

MODERN FORTRAN, PORTABILITY, AND THE RSE-SHAPED HOLE IN SCIENTIFIC
COMPUTING

Fortran is cool

Don't let your friends convince you otherwise

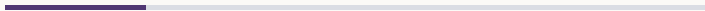
Jorge Luis Gálvez Vallejo (NCI)
jorge.galvezvallejo@anu.edu.au

NCI
Australian National University
Canberra, Australia / August 21, 2026



1

The reputation



1 Your friends are half right

Fortran 77

```
SUBROUTINE AXPY(N,A,X,Y)
COMMON /BLK/ SCRATCH(1000)
DIMENSION X(1),Y(1)
DO 10 I = 1, N
    Y(I) = A*X(I) + Y(I)
10 CONTINUE
RETURN
END
```

Modern Fortran

```
pure subroutine axpy(a, x, y)
    real, intent(in)      :: a, x(:)
    real, intent(inout)   :: y(:)
    y = a*x + y
end subroutine
```

1 Why the debt exists

- These codebases were written by **scientists**, not software engineers
- This is a feature, not a bug
- Climate, weather, quantum chemistry, and CFD still run on this code. It is **not going away**

So

Not a lost cause.

This is the RSE-shaped hole in the talk. I will come back to it.

2

The language



2 Modern Fortran, in one slide

- Object orientation **F2003**
- (Almost) safe dynamic memory —
allocatable, move_alloc
- Arrays as first-class citizens —
slicing, elemental, reductions
- Native parallelism **F2008**
 - `do concurrent` — shared memory + GPU
 - `coarrays` — distributed memory

The point of the whole talk

Fortran lets a domain scientist write **fast** code without first becoming a **expert programmer**.

I use `do concurrent`. For distributed memory I use MPI because it is just better

2 One loop, every device

Standard Fortran

```
do concurrent (k=1:nk,j=1:nj,i=1:ni)
  c(i,j,k) = al*a(i,j,k) + b(i,j,k)
end do
```

The directive wall

```
!$omp target teams loop
  collapse(3)
do k = 1, nk
  do j = 1, nj
    do i = 1, ni
      c(i,j,k) = alpha*a(i,j,k)
      + b(i,j,k)
    end do
  end do
end do
```

2 do concurrent **across compilers**

Each vendor offloads to *its* GPU — NVIDIA is the most mature, hands down

Compiler	DC → CPU	DC → GPU	Notes / flags
nvfortran (NVHPC)	Yes	Yes	most mature; -stdpar=gpu; managed mem; F2023 reduce/locality
ifx (Intel)	Yes	Yes	DC lowered to OpenMP; -fopenmp-target-do-concurrent; ifort EOL
ftn (Cray CCE)	Yes	Yes	AMD + NVIDIA, unified mem; -h thread_do_concurrent
flang/aof (AOCC)	Yes	No	x86/Zen only; -mp; → llvm-flang in 6.0
xlf (IBM XL)	Yes	Part.	POWER+NVLink only; <i>legacy</i> (Open XL)
frt (Fujitsu)	Yes	No	SVE auto-vec; -Kparallel; not DC-aware
gfortran (GCC)	Part.	No	generic autopar -ftree-parallelize-loops=N; not DC-aware
LLVM flang	Part.	Part.	emerging; -fdo-concurrent-to-openmp=host device
lfortran	Part.	Part.	beta; unified mem; not production-ready
nagfor (NAG)	No	No	serial; correctness oracle, not a perf path

3

But does it really work?



3 “Sure — for easy loops”

The objection writes itself. So: the hardest realistic kernel in my field.

- **Two-electron repulsion integrals** — irregular, deeply nested, the kernel everyone insists must be hand-written CUDA
- Prior art: Alkan *et al.*, del Angel — OpenMP target inside GAMESS, **per-shell-quartet**
- **terco**: batches of **shell pairs** dispatched to the device, so the GPU is never starved

OpenACC is used **only** to map data host↔device — unneeded under unified memory. All parallelism is do concurrent.

Result

3.5× faster

a single **V100**

vs. **Q-Chem** on 104 threads (Sapphire Rapids)

3 Why not compare against cuEST?

Because it would not be fair

cuEST is **density-fitted only**. `terco` evaluates **conventional** integrals. Comparing them would flatter me and mislead you.

Q-Chem instead

Commercial, well optimized, same method. A baseline that can actually say no.

3 Two witnesses, four vendors

Code	What it proves
terco	the hardest <i>kernel</i>
Rakali	a whole <i>application</i> , 100% Fortran
metalquicha	a second <i>domain</i> entirely
MOM6	200k lines I <i>did not write</i>

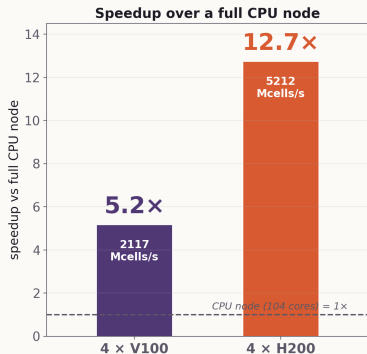
Rakali runs on

GPUs — NVIDIA, AMD, Intel

CPUs — Apple, ARM, Intel, AMD

One source.

Climate and quantum chemistry. Different codebases, same answer.



- Coasts, rivers, floods, open ocean
- Lakes Entrance, Gippsland — 30 m grid
- **5.2x** on 4xV100, **12.7x** on 4xH200, against a full 104-core node

The honest part

Strong scaling: V100 87% efficient, H200 only **69%**. Faster compute **exposes halo communication**. Better hardware made the parallel efficiency worse.

4

The ecosystem gap



4 The turn: the language is portable. The ecosystem is not.

- **stdlib** is genuinely good — and in practice it builds with GNU
- `mpi_f08` needs **vapaa** (Hammond) to cross compilers at all
- **coarrays**: elegant in F2008 — but GCC needs OpenCoarrays, Intel ships its own runtime, Cray's is good, NVIDIA and AMD are nowhere
- Every project reimplements the same five things: sorting, logging, timers, strings, BLAS glue

My criterion is not “native is good”

It is **portable and production-ready, or it doesn't count**. `do concurrent` passes. Coarrays do not — yet.

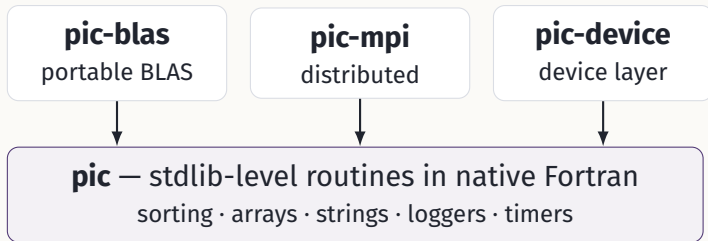
4 A decision I made, out loud

I could put a coarray backend behind `pic-mpi`. I chose not to.

- Most coarray runtimes are **implemented on MPI** anyway — I would be wrapping a wrapper
- I would owe it a **CI matrix** on compilers where coarrays barely work
- The abstraction would make the MPI path *worse* to serve a backend I don't believe in

That is the kind of decision Fortran codebases
~~don't get made for them often enough.~~

4 So I built the missing layer



interop, because you are never trapped: **libfint** (libcint, ported) · **cuEST** module interface

fpm

CMake

builds where the science builds

5

The ask



Three engines, one interface

tblite (semi-empirical, CPU)

libcint (ab initio, CPU)

cuEST (GPU)

All behind `qc_method_t`. Swap the engine —
fragmentation and screening are unchanged.

The CPU path exists to be *disagreed with*

It gives the GPU path a **second, independent implementation** to check against — and every method is validated against **PySCF**.

[CI](#)[coverage](#)[ReadTheDocs](#)[FORD](#)[pre-commit](#)[contributing guide](#)

Differential testing plus an external oracle — exactly what scientific Fortran usually lacks.

5 The ask

Fortran does not need rewriting. It needs engineering.

- CI matrices across six compilers
- Packaging that actually installs
- API design
- Tests, oracles, differential checks
- Documentation that builds
- Reviews

That is RSE work. In Fortran, it is largely **unclaimed**.

5 terco means *stubborn*

Born out of a refusal to accept that something cannot be done in plain old Fortran.

So is this talk.

```
pic · pic-blas · pic-mpi · pic-device  
rakali · metalquicha · terco · libfint  
github.com/JorgeG94 · fortran-lang.org
```

5 Backup: coarrays, if asked

Elegant language design — but distributed memory is the one place where the ecosystem beats the language. MPI is portable today, coarrays are not, and I won't recommend something I wouldn't ship. If the vendors converge, I'll change my mind.

Thank you

Fortran doesn't need rewriting. It needs engineering.

Jorge Luis Gálvez Vallejo (NCI)

jorge.galvezvallejo@anu.edu.au



github.com/JorgeG94/pic

